МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Департамент цифровых, робототехнических систем и электроники

«Элементы объектно-ориентированного программирования в языке Python»

Отчет по лабораторной работе № 2
по дисциплине «Объектно-ориентированное программирование»
Вариант 5

Выполнил студент группы ИВТ-б-о-24-1
Грабарь Артемий Павлович
«__» сентября 2026г.
Подпись студента_____
Работа защищена « »_____20__г.
Проверил Воронкин Р.А. _____
(подпись)

Ставрополь 2026

Цель работы: приобретение навыков по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.

Ссылка на GitHub: https://github.com/Arhi258/OOP_Laba_2.git

Задание:

1. Создать общедоступный репозиторий на GitHub, в котором будет использована лицензия MIT и выбранный язык программирования.
2. Выполнить клонирование созданного репозитория на рабочий компьютер.
3. Дополнить файл .gitignore необходимыми правилами для работы с IDE PyCharm.
4. Создать проект PyCharm в папке репозитория
5. Решить практические задания работы.
6. Зафиксировать сделанные изменения в репозитории.
7. Отправить сделанные изменения на сервер GitHub.

Ход работы:

Был создан общедоступный репозиторий на GitHub с лицензией MIT.

Было выполнено клонирование созданного репозитория на рабочий компьютер.

Был создан проект в среде PyCharm. Для изоляции зависимостей было настроено виртуальное окружение Python. Файл .gitignore был дополнен исключениями для временных файлов и настроек среды разработки. Файл README.md был дополнен сведениями о проекте и командами запуска.

Было выполнено задание 1: «Парой называется класс с двумя полями, которые обычно имеют имена first и second. Требуется реализовать тип данных с помощью такого класса. Во всех заданиях обязательно должны присутствовать: метод инициализации __init__; метод должен контролировать значения аргументов на корректность; ввод с клавиатуры read; вывод на экран. Реализовать внешнюю функцию с именем make_тип(), где тип – тип

реализуемой структуры. Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу. В раздел программы, начинающийся после инструкции if __name__ = ‘__main__’: добавить код, демонстрирующий возможности разработанного класса. Поле first – дробное положительное число, цена товара; поле second – целое положительное число, количество единиц товара. Реализовать метод cost() – вычисление стоимости товара».

Для выполнения задания был реализован класс Pair с полями first и second. В методе __init__ обеспечена проверка корректности цены и количества товара. Методы read и display выполняют ввод и вывод данных. Метод cost вычисляет общую стоимость товара как произведение цены и количества. В отдельном пакете была реализована функция make_pair, предназначенная для создания объекта класса Pair.

Работа класса была проверена на примере товара стоимостью 125,5 и количеством 4 единицы. Полученная общая стоимость составила 502,00. Результат выполнения задания представлен на рисунке 1.

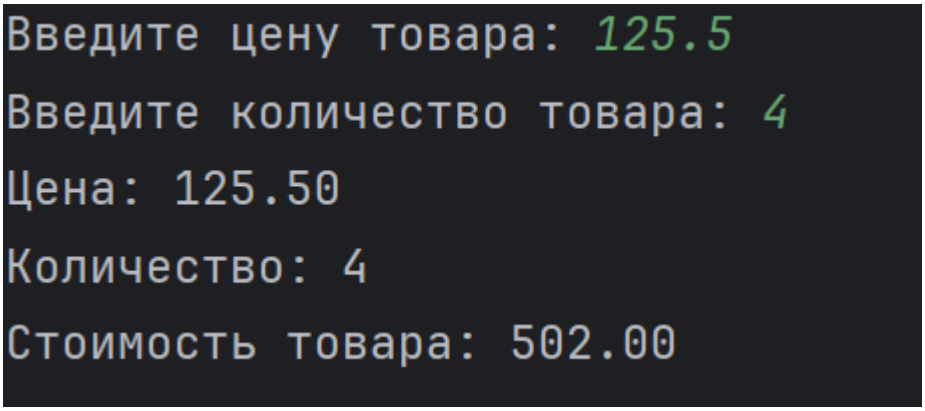


Рисунок 1. Результат выполнения задания 1

Было выполнено задание 2: «Составить программу с использованием классов и объектов для решения задачи. Во всех заданиях, помимо указанных в задании операций, обязательно должны быть реализованы следующие методы: метод инициализации __init__; ввод с клавиатуры read; вывод на экран display. В раздел программы, начинающийся после инструкции if

__name__ = ‘__main__’: добавить код, демонстрирующий возможности разработанного класса. Создать класс Angel для работы с углами на плоскости, задаваемые величиной в градусах и минутах. Обязательно должны быть реализованы: перевод в радианы, приведение к диапазону 0-360, увеличение и уменьшение угла на заданную величину, получение синуса, сравнение углов.»

Для выполнения второго задания был разработан класс Angel для работы с углами, заданными в градусах и минутах. Были реализованы методы инициализации объекта, ввода и отображения данных, перевода углов в радианы, приведения к диапазону от 0 до 360 градусов, увеличения и уменьшения угла, вычисления синуса и сравнения углов.

При выполнении программы первоначальный угол 390° 30′ был приведён к 30° 30′. После увеличения и уменьшения был получен угол 36° 0′. Сравнение с углом 40° 0′ показало, что первый угол меньше второго. Результат выполнения задания представлен на рисунке 2.

```
Введите градусы: 390
Введите минуты: 30
Угол: 390° 30′
В радианах: 6.8155
Синус угла: 0.5075
После приведения к диапазону 0-360°:
Угол: 30° 30′
Увеличение угла:
Введите градусы: 10
Введите минуты: 30
Угол: 41° 0′
Уменьшение угла:
Введите градусы: 5
Введите минуты: 0
Угол: 36° 0′
Введите второй угол для сравнения:
Введите градусы: 40
Введите минуты: 0
Первый угол меньше второго.
```

Рисунок 2. Результат выполнения задания 2

Для проверки корректности работы обоих заданий были созданы автоматические тесты с использованием библиотеки pytest. Тестами были проверены расчёт стоимости товара, обработка некорректных значений, преобразования углов и работа методов классов. По результатам тестирования успешно пройдено 19 проверок. Результат представлен на рисунке 3.

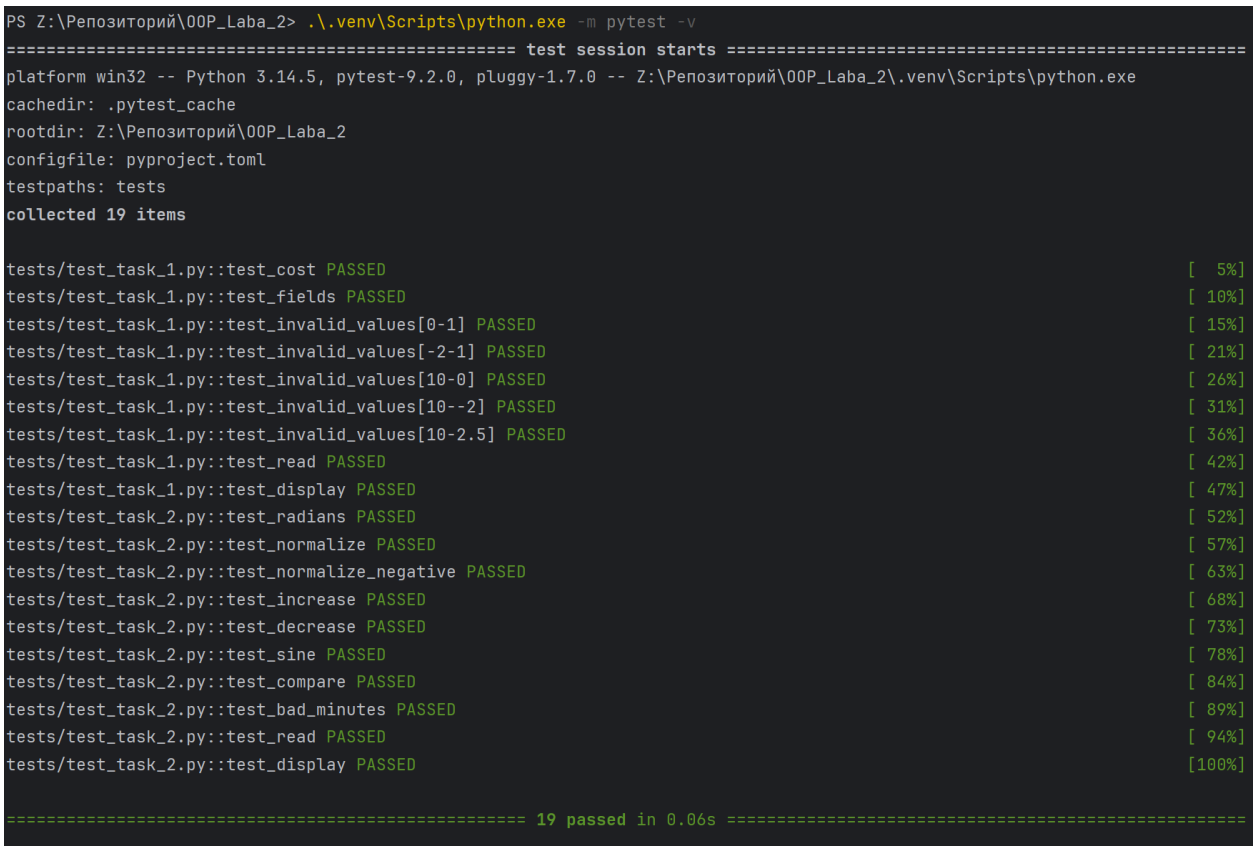


Рисунок 3. Результат автоматического тестирования pytest

Для статического анализа программного кода была использована утилита Ruff. При проверке ошибок оформления импортов и других замечаний не обнаружено. Результат представлен на рисунке 4.



Рисунок 4. Результат проверки программного кода Ruff

Дополнительно была проведена проверка форматирования программного кода с использованием Ruff. Все 14 проверенных файлов соответствовали заданным требованиям форматирования. Результат представлен на рисунке 5.



Рисунок 5. Результат проверки форматирования Ruff

Все изменения были зафиксированы и отправлены на сервер GitHub.

Контрольные вопросы:

1. Классы объявляются с помощью ключевого слова class, после которого указывается имя класса.

2. Атрибуты класса принадлежат самому классу и могут использоваться всеми его объектами. Атрибуты экземпляра относятся к конкретному объекту и обычно задаются через self в методе __init__.

3. Методы представляют собой функции, объявленные внутри класса. Они определяют действия, которые могут выполнять объекты этого класса.

4. Метод __init__ автоматически вызывается при создании экземпляра класса и используется для его инициализации. В нём обычно задаются начальные значения атрибутов объекта.

5. Параметр self обозначает конкретный экземпляр класса и позволяет обращаться к его атрибутам и методам.

6. Атрибуты класса задаются в теле класса, а атрибуты экземпляра обычно создаются присваиванием через self.

7. В Python нет строгого запрета на обращение к атрибутам по их имени. Для обозначения внутренних атрибутов используют соглашение с одним подчёркиванием, а для контролируемого доступа могут применяться свойства property.

8. Функция isinstance(obj, Cls) проверяет, является ли объект obj экземпляром класса Cls или его подкласса. Она применяется для проверки типов объектов.

Вывод: В результате работы были получены навыки по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.

Приложение

Приложение 1.

Код файла Tasks/task_1/pair/__init__.py

```
from .module import Pair

__all__ = ["Pair"]
```

Код файла Tasks/task_1/pair/module.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math



class Pair:
    def __init__(self, first, second):
        if (
            isinstance(first, bool)
            or not isinstance(first, (int, float))
            or not math.isfinite(first)
            or first <= 0
        ):
            raise ValueError("Цена должна быть положительным числом.")

        if type(second) is not int or second <= 0:
            raise ValueError("Количество должно быть положительным целым числом.")

        self.first = float(first)
        self.second = second
```

```
    @staticmethod  # read() можно вызвать через класс
    def read():
        first = float(input("Введите цену товара: "))
        second = int(input("Введите количество товара: "))
        return first, second

    def display(self):
        print(f"Цена: {self.first:.2f}")
        print(f"Количество: {self.second}")

    def cost(self):
        return self.first * self.second
```

Код файла Tasks/task_1/make_pair/__init__.py

```
from .module import make_pair

__all__ = ["make_pair"]
```

Код файла Tasks/task_1/make_pair/module.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from Tasks.task_1.pair import Pair



def make_pair(first, second):
    try:
        return Pair(first, second)
    except ValueError as error:
        print(f"Ошибка: {error}")
        raise SystemExit(1) from None
```

Код файла Tasks/task_1/main.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from Tasks.task_1.make_pair import make_pair
from Tasks.task_1.pair import Pair



def main():
    try:
        first, second = Pair.read()
    except ValueError:
        print("Ошибка: цена должна быть числом, а количество — целым числом.")
        raise SystemExit(1) from None

    product = make_pair(first, second)
    product.display()
    print(f"Стоимость товара: {product.cost():.2f}")



if __name__ == "__main__":
    main()
```

Приложение 2.

Код файла Tasks/task_2/angel/__init__.py

```
from .module import Angel

__all__ = ["Angel"]
```

Код файла Tasks/task_2/angel/module.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
import math


class Angel:
    def __init__(self, degrees, minutes):
        if type(degrees) is not int or type(minutes) is not int:
            raise ValueError("Градусы и минуты должны быть целыми числами.")

        if not 0 <= minutes < 60:
            raise ValueError("Минуты должны быть от 0 до 59.")

        if degrees < 0:
            self.total_minutes = degrees * 60 - minutes
        else:
            self.total_minutes = degrees * 60 + minutes

    @staticmethod
    def read():
        degrees = int(input("Введите градусы: "))
        minutes = int(input("Введите минуты: "))
        return degrees, minutes

    def display(self):
        sign = "-" if self.total_minutes < 0 else ""
        degrees, minutes = divmod(abs(self.total_minutes), 60)
        print(f"Угол: {sign}{degrees}° {minutes}′")

    def to_radians(self):
        return math.radians(self.total_minutes / 60)
```

```
    def normalize(self):
        self.total_minutes %= 360 * 60

    def increase(self, degrees, minutes):
        if type(degrees) is not int or degrees < 0:
            raise ValueError(
                "Добавляемые градусы должны быть целым неотрицательным
числом."
            )

        amount = Angel(degrees, minutes)
        self.total_minutes += amount.total_minutes

    def decrease(self, degrees, minutes):
        if type(degrees) is not int or degrees < 0:
            raise ValueError(
                "Вычитаемые градусы должны быть целым неотрицательным
числом."
            )

        amount = Angel(degrees, minutes)
        self.total_minutes -= amount.total_minutes

    def sine(self):
        return math.sin(self.to_radians())

    def compare(self, other):
        if not isinstance(other, Angel):
            raise ValueError("Для сравнения нужен объект Angel.")
```

```
        first = self.total_minutes % (360 * 60)
        second = other.total_minutes % (360 * 60)

        if first > second:
            return 1
        if first < second:
            return -1
        return 0
```

Код файла Tasks/task_2/main.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from Tasks.task_2.angel import Angel


def main():
    try:
        degrees, minutes = Angel.read()
        angle = Angel(degrees, minutes)

        angle.display()
        print(f"В радианах: {angle.to_radians():.4f}")
        print(f"Синус угла: {angle.sine():.4f}")

        angle.normalize()
        print("После приведения к диапазону 0–360°:")
        angle.display()

        print("Увеличение угла:")
```

```
        degrees = int(input("Введите градусы: "))
        minutes = int(input("Введите минуты: "))
        angle.increase(degrees, minutes)
        angle.display()

        print("Уменьшение угла:")
        degrees = int(input("Введите градусы: "))
        minutes = int(input("Введите минуты: "))
        angle.decrease(degrees, minutes)
        angle.display()

        print("Введите второй угол для сравнения:")
        degrees, minutes = Angel.read()
        other = Angel(degrees, minutes)

        result = angle.compare(other)

        if result > 0:
            print("Первый угол больше второго.")
        elif result < 0:
            print("Первый угол меньше второго.")
        else:
            print("Углы равны.")

    except ValueError as error:
        print(f"Ошибка: {error}")
        raise SystemExit(1) from None


if __name__ == "__main__":
```

```
    main()
```

Приложение 3.

Код файла tests/test_task_1.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import pytest

from Tasks.task_1.make_pair import make_pair
from Tasks.task_1.pair import Pair


def test_cost():
    product = make_pair(12.5, 4)
    assert product.cost() == 50.0


def test_fields():
    product = Pair(20, 3)
    assert product.first == 20.0
    assert product.second == 3


@pytest.mark.parametrize(
    "price, quantity",
    [
        (0, 1),
        (-2, 1),
        (10, 0),
        (10, -2),
```

```
        (10, 2.5),
    ],
)
def test_invalid_values(price, quantity):
    with pytest.raises(SystemExit):
        make_pair(price, quantity)


def test_read(monkeypatch):
    answers = iter(["25.5", "2"])
    monkeypatch.setattr(
        "builtins.input",
        lambda _: next(answers),
    )

    assert Pair.read() == (25.5, 2)


def test_display(capsys):
    Pair(3.5, 2).display()
    output = capsys.readouterr().out

    assert "Цена: 3.50" in output
    assert "Количество: 2" in output
```

Код файла tests/test_task_2.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math
```

```
import pytest

from Tasks.task_2.angel import Angel


def test_radians():
    assert Angel(180, 0).to_radians() == pytest.approx(math.pi)


def test_normalize():
    angle = Angel(390, 30)
    angle.normalize()
    assert angle.total_minutes == 30 * 60 + 30


def test_normalize_negative():
    angle = Angel(-30, 0)
    angle.normalize()
    assert angle.total_minutes == 330 * 60


def test_increase():
    angle = Angel(30, 15)
    angle.increase(10, 30)
    assert angle.total_minutes == 40 * 60 + 45


def test_decrease():
    angle = Angel(50, 30)
    angle.decrease(10, 45)
```

```
    assert angle.total_minutes == 39 * 60 + 45


def test_sine():
    assert Angel(30, 0).sine() == pytest.approx(0.5)


def test_compare():
    assert Angel(90, 0).compare(Angel(45, 0)) == 1
    assert Angel(30, 0).compare(Angel(60, 0)) == -1
    assert Angel(390, 0).compare(Angel(30, 0)) == 0


def test_bad_minutes():
    with pytest.raises(ValueError):
        Angel(10, 60)


def test_read(monkeypatch):
    answers = iter(["45", "15"])
    monkeypatch.setattr(
        "builtins.input",
        lambda _: next(answers),
    )

    assert Angel.read() == (45, 15)


def test_display(capsys):
    Angel(45, 15).display()
```

```
assert "45° 15′" in capsys.readouterr().out
```